

Relatório de Conversão NCL → Python

Modelo de Risco de Fogo Previsto — INPE_FireRiskModel v2.2

4 de agosto de 2026

Scripts convertidos: rf_previsto_1-5dias_2023.sh e rf_previsto_1-2_semanas_2024.sh

Modelo: Alberto Setzer (INPE) · Código NCL original: Guilherme Martins (INPE)

Programa Queimadas — <http://www.inpe.br/queimadas/>

Sumário

1. Objetivo
2. Arquitetura original
3. Arquitetura convertida
4. Formulação do modelo (inalterada)
5. Tabela de substituições
6. Validação
7. Diferenças de comportamento e melhorias
8. Pontos de atenção
9. Conclusão

1. Objetivo

Este relatório documenta a conversão dos dois scripts operacionais de cálculo do Risco de Fogo (RF) previsto em 1 km, originalmente escritos em Bash com o núcleo de cálculo em NCL (NCAR Command Language), para Python. A motivação principal é a descontinuação do NCL pelo NCAR, que o mantém apenas em modo de manutenção, e a consolidação da cadeia de processamento em uma única linguagem com ecossistema ativo.

A conversão eliminou todas as dependências do ambiente `conda ncl_stable`: o interpretador `ncl`, o `cdo`, o `gdal_translate` (binário) e o `GNU parallel`. Todo o processamento passa a ser feito com bibliotecas Python de uso amplo na comunidade científica: `numpy`, `xarray`, `netCDF4` e `rasterio`.

2. Arquitetura original

Cada script original operava em quatro etapas. Primeiro, montava em arquivos temporários a lista dos 119 arquivos diários de precipitação observada do IMERG mais um arquivo de precipitação prevista do GFS. Em seguida, gerava dinamicamente (via heredoc) um script NCL por horário de previsão e os executava em paralelo com o `GNU parallel`. Depois, aguardava a produção dos arquivos com um laço de espera (`sleep 600`), corrigia o eixo de tempo e o valor ausente de cada `NetCDF` com o `cdo` e convertia para `GeoTIFF` com o `gdal_translate`. Por fim, publicava os resultados no geoserver TerraBrasilis (script auxiliar, `lftp` e `scp`).

As principais diferenças entre os dois scripts:

Característica	1 a 5 dias (2023)	1 a 2 semanas (2024)
Horizontes de previsão	+6 h até +4 dias 18 UTC, a cada 6 h	+7 e +14 dias, às 18 UTC
Número de previsões	19	2
Risco básico máximo (rb)	0.9	0.8
Paralelismo	<code>parallel -j 20</code>	<code>parallel -j 2</code>
Particularidade	Fogograma (mergetime) e envio via <code>envia_geoserv_tbrasilis.sh</code>	Cópia dos arquivos GFS de 12 UTC para 18 UTC no dia +14; envio via <code>lftp</code> e <code>scp</code>
Diretório de saída	<code>RF_PREV</code>	<code>RF_PREV_SEMANAL</code>

3. Arquitetura convertida

A versão Python é composta por quatro programas e dois testes:

Arquivo	Papel
<code>rf_core.py</code>	Núcleo do cálculo do RF, compartilhado por todos os scripts. Substitui o script NCL gerado via heredoc, o <code>cdo</code> e o <code>gdal_translate</code> .
<code>rf_previsto_1_5dias.py</code>	Orquestrador equivalente ao <code>rf_previsto_1-5dias_2023.sh</code> .
<code>rf_previsto_1_2_semanas.py</code>	Orquestrador equivalente ao <code>rf_previsto_1-2_semanas_2024.sh</code> .
<code>rf_previsto.py</code>	Script genérico: calcula o RF para qualquer horizonte de previsão (lista ou intervalo, inclusive meses) e qualquer fonte de dados (<code>--fonte gfs eta besm</code>), com <code>rb</code> máximo, produto e diretório base configuráveis e fallback opcional do GFS.
<code>rf_fontes.py</code>	Camada de fontes de previsão: configuração por fonte (padrões de nome,

Arquivo	Papel
	variáveis, frequência dos acúmulos 1h/12h/1d, unidades, alcance), agregação para o passo diário e montagem da série de 120 dias misturando IMERG observado e precipitação prevista.
teste_rf.py	Teste de validação com dados sintéticos e comparação com implementação de referência fiel ao NCL.
teste_rf_previsto.py	Teste de ponta a ponta do script genérico com a estrutura de diretórios da produção reproduzida em /tmp.
teste_rf_multifonte.py	Teste do modo multifonte: Eta a 13 meses, BESM com acúmulos de 12 h, fonte via JSON com frequência de 1 h e equivalência numérica com o núcleo.

A geração de scripts NCL temporários foi substituída por uma chamada de função (rf_core.calcula_risco_fogo), e o parallel por um ProcessPoolExecutor da biblioteca padrão, com o mesmo número de processos dos scripts originais (configurável por --jobs). O laço de espera com sleep 600 tornou-se desnecessário: o pool de processos é controlado diretamente e, se alguma previsão falhar, o erro é registrado em log e o script termina com código de saída 1, preservando o contrato do original.

4. Formulação do modelo (inalterada)

A formulação física do RF foi preservada exatamente como no NCL:

- Precipitação acumulada** — a série de 120 tempos (119 dias de IMERG + 1 previsão do GFS) é invertida no tempo e acumulada olhando para trás: $vprec(i) = vprec(i-1) + precip(i)$.
- Fatores de precipitação (fp)** — onze fatores exponenciais $fp = \exp(cte \times \Delta vprec)$ com as constantes -0.14, -0.07, -0.04, -0.03, -0.02, -0.01, -0.008, -0.004, -0.002, -0.001 e -0.0007, cobrindo as janelas de 1, 2, 3, 4, 5, 6-10, 11-15, 16-30, 31-60, 61-90 e 91-120 dias.
- Dias de secura (PSE)** — $PSE = 105 \times fp_1 \times fp_2 \times \dots \times fp_{11}$, interpolado bilinearmente da grade da precipitação (~10 km) para a grade de 1 km do mapa de vegetação.
- Risco básico (rb)** — por classe de vegetação, com $A = (-999.9, 6, 4, 3, 2.4, 2, 1.72, 1.5)$ e $PSE_max = (-999.9, 30, 45, 60, 75, 90, 105, 120)$. Se $PSE > PSE_max$ da classe, $rb = rb_max$ (0.9 ou 0.8); caso contrário, $rb = (rb_max \times (1 + \sin((A \times PSE - 90) \times 3.1416/180))) / 2$. A classe 0 (superfícies líquidas) é valor ausente.
- Fatores meteorológicos** — Fator de Umidade $FU = -0.008 \times UR + 1.3$ (UR em décimos; coeficiente alterado de -0.006 para -0.008 em 11/04/2019 para incluir a sazonalidade) e Fator de Temperatura $FT = 0.02 \times T + 0.4$ (T em °C), ambos interpolados para 1 km.
- Risco final** — $rbf = rb \times FT \times FU$, limitado a 1; corrigido pelo Fator de Latitude $FLAT = 1 + |lat| \times 0.003$ e pelo Fator Topográfico $FTOP = 1 + elev \times 0.00003$ (equações de Setzer, 05/04/2019), novamente limitado a 1 e arredondado a 2 casas decimais.

Observação de fidelidade: o NCL usa a constante 3.1416 (e não π) na conversão para radianos da equação do rb. Esse valor foi mantido deliberadamente para reproduzir os resultados operacionais.

5. Tabela de substituições

NCL / ferramenta externa	Equivalente em Python
addfile / addfiles + ListSetType("cat")	xarray.open_dataset + numpy.concatenate

NCL / ferramenta externa	Equivalente em Python
precip(:,:,1,:,:) — inversão do tempo	precip[::-1, :, :, :]
Laço do while da precipitação acumulada	numpy.cumsum(precip, axis=0)
exp, sin, abs, where	numpy.exp, numpy.sin, numpy.abs, numpy.where
linint2_Wrap (interpolação bilinear)	rf_core.interp_bilinear (numpy vetorizado)
Duplo laço do i / do k do risco básico	Indexação vetorizada A_VEG[veg], PSE_MAX_VEG[veg]
floattoint(mapa_veg)	Conversão para int32 na leitura
mapa_veg@_FillValue = 0.0	Classe 0 → NaN (índice 0 dos vetores A / PSE_max)
conform_dims (FLAT 1D → 2D)	Broadcasting FLAT[:, np.newaxis]
fspan(latS, latN, n)	numpy.linspace
decimalPlaces(rbf, 2, True)	numpy.round(rbf, 2)
copy_VarCoords	Coordenadas explícitas do xarray.Dataset
Escrita NetCDF (addfile "c", fileattdef, filedimdef)	xarray.Dataset.to_netcdf (NETCDF4_CLASSIC, time ilimitado)
cdo -r -setmissval,-999 -setaxis,...	Eixo de tempo e _FillValue -999 definidos na gravação
cdo -O mergetime	rf_core.mergetime (xarray.concat + sortby)
gdal_translate (GTiff, EPSG:4326, TILED, LZW)	rf_core.netcdf_para_geotiff (rasterio)
parallel -j N + scripts .ncl temporários	ProcessPoolExecutor(max_workers=N)
Heredoc cat << EOF > rf.prev.*.ncl	Chamada direta de rf_core.calcula_risco_fogo
Laço de espera (sleep 600 + contagem)	Controle direto do pool + verificação final + exit 1
lftp / scp / envia_geoserv_tbrasilis.sh	subprocess.run (desativável com --sem-envio)

6. Validação

O teste teste_rf.py constrói um cenário sintético completo (119 arquivos IMERG + GFS de precipitação e de temperatura/umidade + mapa de vegetação + topografia) e valida a cadeia de ponta a ponta:

Verificação	Resultado
RF final vetorizado vs. implementação de referência com laços explícitos idêntica ao NCL	erro máximo $\approx 3 \times 10^{-8}$
interp_bilinear vs. scipy.RegularGridInterpolator	erro máximo $\approx 2 \times 10^{-6}$
Classe de vegetação 0 (água)	valor ausente (NaN), como no NCL
Faixa de valores do RF	[0, 1]
Eixo de tempo do NetCDF de saída	data/hora da previsão, time ilimitado, _FillValue -999
GeoTIFF	EPSG:4326, nodata -999, LZW, tiled, orientação norte→sul, limites corretos
mergetime	horários concatenados e ordenados corretamente
Leitura do IMERG real (IMERG.YYYYMMDD.nc)	variável prec, grade 0.1°, 901×850 pontos, lat -60.05...29.95, lon -114.95...-30.05

7. Diferenças de comportamento e melhorias

1. **Desempenho** — o duplo laço do NCL sobre a grade de 1 km (dezenas de milhões de pontos) foi vetorizado em numpy; o tempo de cálculo por previsão cai de forma expressiva em relação às ~3 h totais originais.
2. **Arquivos temporários** — arquivo.prev.prec.*, paralelizar_RF_PREV.txt e os scripts .ncl gerados dinamicamente deixam de existir; as listas são montadas em memória.
3. **Tratamento de falhas** — o NCL abortaria com erro de índice se houvesse menos de 120 tempos de precipitação; a versão Python valida explicitamente e registra mensagem clara no log. Arquivos IMERG faltantes continuam listados em log.falta.arquivos.prev.prec.txt.
4. **Aviso do NCL suprimido** — a mensagem "warning: error attempting to fix non-monotonic aggregation variable" (inofensiva, causada pela inversão do tempo) não existe mais, pois a concatenação é feita em numpy.
5. **Logs** — cada previsão grava seu log em log/log.<data_modelo>.<data_previsao>, como no original.
6. **Novos parâmetros de linha de comando** — --data-final YYYYMMDD (reprocessamento de datas passadas), --jobs N e --sem-envio (executa sem publicar nos servidores).
7. **Script genérico de horizontes** — o rf_previsto.py (inexistente na versão NCL) permite gerar o RF para qualquer conjunto de horizontes, via --horizontes 18h,2d18h,7d18h,6m ou --de 1m --ate 13m --passo 1m, com --rb-max, --produto, --base, --fallback-gfs, --sem-tif e --fogograma.
8. **Múltiplas fontes de previsão** — com --fonte gfs|eta|besm (e --config-fontes para ajustar padrões de nome, variáveis e frequências via JSON), a série diária de 120 tempos passa a combinar IMERG observado com precipitação prevista da fonte, agregando acúmulos de 1h/12h/1 dia para o passo diário e regradeando para a grade do IMERG. Isso estende o alcance do RF de ~16 dias (GFS) para até 13 meses (Eta e BESM).

8. Pontos de atenção

- **Credenciais do lftp** — o script original continha usuário e senha em texto claro; foram mantidos como constantes no topo de rf_previsto_1_2_semanas.py para não alterar o comportamento, mas recomenda-se fortemente migrá-los para variáveis de ambiente ou ~/.netrc.
- **Memória** — cada processo carrega arrays de 1 km em float64 (na grade completa da América do Sul, ~0.6 GB por array). Com --jobs 20 o consumo agregado é alto, como no original; ajuste --jobs à capacidade da máquina.
- **Caminhos absolutos** — os diretórios de produção (/home/queimadas/INPE_FireRiskModel/...) foram mantidos idênticos e estão centralizados em constantes no topo de cada script.
- **Interpolação nas bordas** — linint2 do NCL retorna valor ausente fora do domínio de origem; a implementação Python usa "clamp" nas bordas. Como a grade de 1 km está contida no domínio da precipitação e do GFS, não há diferença prática.
- **Ano do mapa de vegetação** — mantida a regra original: anos ≥ 2020 usam o mapa de 2019 (Merge_MapBiomas_V5_IGBP_C6_2019.nc).

9. Conclusão

A conversão reproduz a formulação e os produtos dos scripts originais com fidelidade numérica verificada (diferenças da ordem de 10^{-8} , atribuíveis a arredondamento de ponto flutuante), elimina a dependência de NCL, cdo, GDAL (binário) e GNU parallel, reduz o tempo de execução pela vetorização do cálculo do risco básico e melhora a operação com validações explícitas, logs por previsão e parâmetros de linha de comando para reproprocessamento e testes.